IMDb Data Engineering Pipeline

Savin Mihai-Alexandru[1]

Faculty of Computer Science, "Alexandru Ioan Cuza" University of Iași

1 Executive Summary

This project implements an automated, end-to-end data engineering pipeline that extracts raw IMDb datasets, loads them into a DuckDB data warehouse, and transforms them into a query-ready star schema using dbt. Orchestrated by Apache Airflow, the pipeline ultimately answers complex analytical questions regarding top-rated directors, movie runtime evolution, and the ratio of hidden gems to overrated titles within various cinematic genres.

2 Dataset Description

The primary data source consists of bulk exports from the official IMDb website, accessible via `datasets.imdbws.com`.

- **Source, Size, and Frequency:** The dataset contains millions of rows distributed across seven large, compressed TSV files, which IMDb updates on a daily basis.
- **Scoping Decisions:** The extraction process retrieves all seven core files provided by the platform (`name.basics`, `title.akas`, `title.basics`, `title.crew`, `title.episode`, `title.principals`, and `title.ratings`) to ensure a complete foundation for the data warehouse before any targeted transformations occur.

2.1 Raw Data Structure

Before being processed and cleaned in dbt, the original non-commercial dataset operates on a relational structure centered around two main pillars: **Productions** (identified by the `tconst` primary key) and **Persons** (identified by the `nconst` primary key).

The Productions pillar includes tables such as `title.basics` (core movie and show metadata), `title.akas` (regional alternative titles), `title.ratings` (user scores and votes), and `title.episode` (TV series hierarchy). The Persons pillar is anchored by `name.basics`, which contains biographical data for actors, directors, and crew members.

These two domains are closely linked via intersection bridge tables, notably `title.crew` (providing preliminary director and writer lists) and `title.principals` (detailing the exact cast and crew billing, including character names).

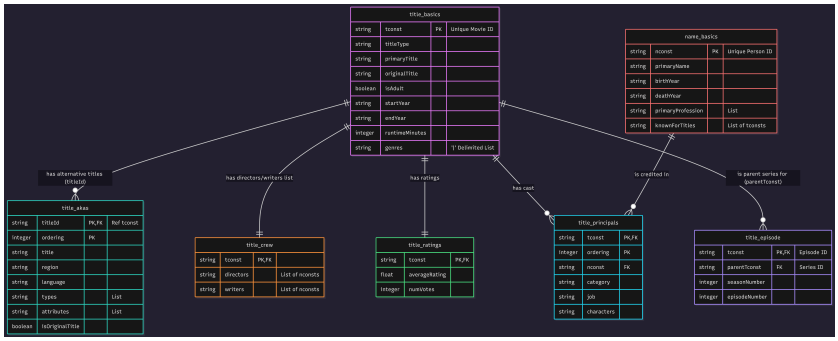

Fig. 1. Entity-Relationship Diagram of the raw IMDb dataset

Figure 1 illustrates the Entity-Relationship Diagram of this raw dataset prior to its dimensional modeling.

To better understand the extraction and loading phases, the raw tables are detailed below based on their specific domains:

The Productions Pillar

- `title.basics`**:** The core table for all productions. It includes the unique identifier (`tconst`), format (`titleType`), release names, runtime, and a delimited list of genres.
- `title.akas`**:** Stores alternative regional titles and translations, linking back to the main movie via `titleId`. It includes localization details such as region and language.
- `title.ratings`**:** The table storing user scores, containing the movie ID, the average score (`averageRating`), and the total number of votes.
- `title.episode`**:** Used to map specific TV episodes (`tconst`) to their parent series (`parentTconst`), alongside season and episode numbers.

The Persons Pillar

- `name.basics`**:** The foundational table for industry personnel. It holds the unique person identifier (`nconst`), credited name, birth/death years, primary professions, and a denormalized list of titles they are known for.

Intersection (Bridge) Tables

- `title.crew`**:** A preliminary relational link providing a comma-separated list of director and writer IDs for a specific title.
- `title.principals`**:** A highly detailed bridge table outlining the exact roles individuals had in a production. It links `tconst` and `nconst`, specifying the billing order, role category (e.g., actor, director), and a JSON array of character names played.

3 Architecture Overview

The project's architecture integrates three main technologies, operating on a scheduled basis to ensure data freshness:

- **Apache Airflow (Orchestrator):** Acts as the central scheduler and trigger mechanism. The DAG is designed to run sequentially, ensuring dependencies are strictly respected.
- **DuckDB (Data Warehouse) & Python:** A Python script, triggered by Airflow, handles the extraction of the raw `.tsv.gz` files. To optimize storage and query performance, these files are first converted into Parquet format. Only the resulting Parquet files are then loaded into the DuckDB database as raw tables.
- **dbt (Transformer):** Once the data lands in DuckDB, Airflow triggers dbt to execute the data modeling phases. dbt handles type casting, filtering, table joins, data testing, and snapshotting, ultimately constructing the final query-ready star schema.

4 Data Modeling Explanation

To meet the analytical requirements of the project, the highly denormalized raw data was transformed into a robust star schema using dbt. This dimensional architecture optimizes query performance, resolves complex many-to-many relationships, and ensures accurate metric aggregations.

4.1 Fact Table and Grain

The core of the model is the `fct_title_ratings` table.

- **Grain:** The grain is strictly defined as *one row per title* (uniquely identified by `title_id` / `tconst`).
- **Justification for Grain:** This explicit grain is critical because movie titles inherently have many-to-many relationships with both genres and industry personnel. If these relationships were flattened directly into the fact table, the row count would explode, leading to duplicated metrics and inaccurate statistical aggregations (e.g., counting a single movie's votes multiple times for every actor in it). The fact table isolates purely quantitative metrics: `average_rating` and `num_votes`.

4.2 Dimension Tables

The dimension tables provide the descriptive context necessary for filtering and grouping the facts:

- `dim_title_snapshot` **(SCD Type 2):** This dimension tracks historical changes in core movie metadata. IMDb periodically corrects or reclassifies attributes such as `runtime_minutes` or `genres`. By implementing a Slowly Changing Dimension (SCD Type 2) via dbt snapshots, the model generates surrogate keys (`dbt_scd_id`) and tracking timestamps (`dbt_valid_from`, `dbt_valid_to`, `is_active`). This preserves historical metadata states as they were at the time of each load, rather than silently overwriting them.
- `dim_person`**:** The central dimension for industry personnel, built by exploding the denormalized `knownForTitles` array from the raw data. It stores the `primary_name` and `birth_year`.
- `dim_year` **&** `dim_genre`**:** Helper dimensions used to accelerate global aggregations, such as grouping movie runtimes by decade or normalizing genre names for comparative analysis.

4.3 Bridge Tables (Resolving Many-to-Many Relationships)

To maintain the strict one-row-per-title grain in the fact table while supporting complex filtering by cast or category, the schema implements two intersection (bridge) tables:

- `bridge_title_genres`**:** Splits the original pipe-delimited genre strings into a normalized mapping between titles and individual genre IDs.
- `bridge_title_crew`**:** Unifies the raw `title.crew` and `title.principals` files into a single, cohesive table. It links a title (`title_id`) to a person (`name_id`) and explicitly defines their `role` (e.g., director, actor, writer).

Figure 2 illustrates the final Star Schema design, demonstrating how the central fact table connects to the surrounding dimensions and bridge tables.

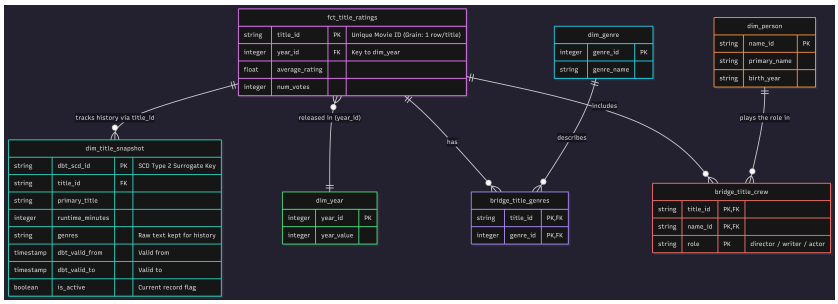

Fig. 2. Entity-Relationship Diagram of the finalized IMDb Star Schema

5 Project Structure Walkthrough

The repository is organized to separate orchestration, ingestion, transformation, and analytical outputs cleanly. Below is a walkthrough of the folder layout and the purpose of each directory:

6 Project Structure Walkthrough

The repository is organized to separate orchestration, ingestion, transformation, and analytical outputs cleanly. Below is a walkthrough of the folder layout and the purpose of each directory:

- `airflow/` - Contains Apache Airflow orchestration files, including the DAG definitions (`dags/`) and Python dependencies (`requirements.txt`), to manage pipeline execution.
- `analyses/` - Stores the final business analytical SQL queries used to answer the project's core deliverable questions.
- `data/` - Acts as the landing directory for the raw IMDb datasets (`.tsv.gz` files) prior to their extraction and ingestion.
- `dbt_packages/` - Contains external dbt packages downloaded as dependencies for the data transformation models.
- `logs/` - Stores execution logs for Airflow tasks and the overall data pipeline, essential for troubleshooting failures.
- `macros/` - Houses reusable dbt macros for custom SQL functions (e.g., extracting decades) and custom testing logic (`tests/`) to keep the codebase DRY (Don't Repeat Yourself).
- `models/` - The core dbt transformation folder, sequentially structured into `staging/` (cleaning, casting, and null handling), `intermediate/` (data reshaping steps, such as unnesting arrays, required before dimensional modeling), and `marts/` (the final query-ready Fact and Dimension tables).
- `scripts/` - Holds Python utility scripts, such as `load_imdb.py` for DuckDB ingestion and `generate_charts.py` for producing analytical visualizations.
- `seeds/` - Contains dbt seed files, which are static mapping CSVs loaded directly into the database as reference tables.
- `snapshots/` - Houses dbt snapshot definitions used to implement SCD Type 2 tracking for slowly changing dimensions (e.g., `dim_title_snapshot`).
- `tests/` - Stores custom dbt data tests to ensure data quality, referential integrity, and business rule enforcement across the pipeline.
- `visualizations/` - The output directory for the auto-generated analytical plots (`.png` files) created by the Python charting script.
- `warehouse/` - The destination folder for the compiled local `imdb.duckdb` database file, acting as the analytical data warehouse.
- `.env / .env.example` - Configuration files used for securely managing environment variables required by Docker and Airflow.
- `db_original.md / db_star.md` - Markdown documentation detailing the schema definitions of the raw IMDb data and the final Star Schema design.

7 Airflow DAG Explanation

The pipeline is orchestrated by Apache Airflow via a Directed Acyclic Graph (DAG) named `imdb_pipeline`.

7.1 Schedule and Configuration

The DAG is configured to run on a `@daily` schedule, ensuring the data warehouse stays synchronized with IMDb's daily updates. To ensure fault tolerance and meet operational standards, the DAG configuration includes a default retry policy of one attempt (`retries: 1`) with a one-minute delay upon task failure.

7.2 Task Dependencies and Execution Flow

The pipeline executes sequentially, strictly enforcing dependencies to ensure data integrity before proceeding to the next logical stage. Figure 3 displays the Graph View of the DAG from the Airflow UI.

Fig. 3. Airflow Graph View illustrating the sequential execution flow of the IMDb pipeline

The logical stages and their respective tasks are defined as follows:

1. `start` **(EmptyOperator):** A dummy task marking the beginning of the DAG execution.
2. `load_imdb_to_bronze` **(PythonOperator):** Handles the Extraction and Loading phase. It reads the raw TSV files, converts them into Parquet format for optimized storage, and loads them directly into DuckDB's `bronze` schema.
3. `validate_bronze_tables` **(PythonOperator):** A preliminary validation step designed to fail fast. It verifies that the DuckDB bronze tables were successfully populated and are not empty before beginning resource-intensive transformations.
4. `dbt_run_staging` **(BashOperator):** Initiates the dbt transformation phase by running models in the `staging/` directory, handling column renaming, type casting, and null handling.
5. `dbt_run_intermediate` **(BashOperator):** Executes the models in the `intermediate/` directory, which apply necessary data reshaping logic (such as unnesting arrays) to prepare the data for dimensional modeling.
6. `dbt_snapshot` **(BashOperator):** Runs the dbt snapshot process to capture Slowly Changing Dimension (SCD Type 2) modifications on the `dim_title` metadata.

7. `dbt_run_marts` **(BashOperator):** Executes the final dimensional models in the `marts/` directory, joining dimensions and facts into the final query-ready star schema.
8. `dbt_test` **(BashOperator):** The final validation phase. It runs all generic schema tests (e.g., uniqueness, non-null) and custom data tests across the dbt project to guarantee data quality and referential integrity.
9. `finish` **(EmptyOperator):** A dummy task signaling the successful completion of the pipeline.

The execution sequence is strictly linear: `start` → `load` → `validate` → `staging` → `intermediate` → `snapshot` → `marts` → `test` → `finish`.

8 dbt Lineage and Tests

8.1 Data Lineage

The dbt lineage graph provides a visual representation of the data flow, illustrating the dependencies from the raw DuckDB `bronze` tables through the `staging` and `intermediate` layers, culminating in the final `marts` dimensional models. Figure 4 shows the full Directed Acyclic Graph (DAG) of the dbt project.

Fig. 4. dbt docs lineage graph showing the transformation flow

8.2 Implemented Data Tests

– **Primary Key Tests (`unique` and `not_null`):** Applied to the primary keys of all staging and mart tables (e.g., `title_id` in `dim_title_snapshot` and

`fct_title_ratings`).
Reason: Ensures that no duplicate records exist and that foundational identifiers are never missing.
- **Referential Integrity Tests (`relationships`):** Applied to the foreign keys in the fact and bridge tables, validating that every `title_id`, `name_id`, or `year_id` explicitly exists in their respective dimension tables.
Reason: Prevents orphaned records in the data warehouse and guarantees that analytical aggregations group correctly across dimensions.
- **Domain Rule: Rating Range (Custom Test):** A custom SQL test applied to `average_rating` in the fact table, ensuring values fall strictly between 0.0 and 10.0.
Reason: Enforces raw business logic; IMDb ratings cannot mathematically exceed 10 or drop below 0. Values outside this range indicate data corruption.
- **Domain Rule: Valid Release Year (Custom SQL):** A custom validation applied to the `dim_year` / `startYear` logic ensuring the year is not set in the distant future.
Reason: Prevents manual data entry typos in the source system from corrupting time-based historical analyses.
- **Expected Failures / Warning Tests (`severity: warn`):** Applied to nullable descriptive fields, such as missing `runtimeMinutes` for obscure indie titles, or missing `genres`.
Reason: Some titles in the raw dataset legitimately lack full metadata. Failing the pipeline on these rows would unnecessarily discard valid rating facts, so a warning is logged instead to maintain pipeline resilience.

9 How to Run

To ensure reproducibility, the pipeline can be executed either fully automated via Docker and Apache Airflow, or manually via a local terminal. Both methods require starting from a clean DuckDB environment.

9.1 Prerequisites: Getting the Raw Data

Before executing the pipeline, the raw datasets must be acquired:

1. Navigate to the official IMDb Datasets page (`https://datasets.imdbws.com/`).
2. Download the following 7 compressed files (`.tsv.gz`): `name.basics`, `title.akas`, `title.basics`, `title.crew`, `title.episode`, `title.principals`, and `title.ratings`.
3. Place all downloaded files directly into the `data/` folder in the project root.

9.2 Execution via Airflow (Docker)

This is the recommended approach for full orchestration:

1. **Configure Environment:** Copy the environment variables template to create the local configuration file.
   - *Linux / macOS / WSL:* `cp .env.example .env`
   - *Windows (CMD / PowerShell):* `copy .env.example .env`
2. **Clean the Environment:** Delete any existing `warehouse/imdb.duckdb` file to ensure a clean state.
3. **Start Containers:** Spin up the Docker containers by running: `docker compose up -d`.
4. **Trigger DAG:** Access the Airflow UI at `http://localhost:8084` (credentials: `admin` / `admin`). Unpause and manually trigger the `imdb_pipeline` DAG to execute all stages sequentially.

9.3 Local Execution (Without Airflow)

To run the pipeline directly in a local terminal for debugging or testing:

1. **Set up the Python Environment:** Create and activate a virtual environment, then install dependencies:

```
python -m venv .venv
source .venv/bin/activate
pip install -r airflow/requirements.txt
pip install duckdb pandas matplotlib seaborn
```

2. **Clean the State:** Delete the `warehouse/imdb.duckdb` file.
3. **Extract & Load:** Run the extraction script to build the DuckDB Bronze layer: `python scripts/load_imdb.py`.
4. **Transform & Test:** Execute the transformation and testing layers via dbt in the root directory by running: `dbt build`.

10 Answers to the Deliverable Questions

10.1 Q1: Top 10 Directors

Question: Who are the top 10 directors by average rating, restricted to those with 5+ titles and 1,000+ combined votes?
Model/Query Used: `q1_v3_top_directors_movies_only.sql`
Result: After filtering specifically for feature films, established Hollywood directors such as Christopher Nolan and Quentin Tarantino emerged at the top of the list.
Interpretation: Filtering strictly for `title_type = 'movie'` is critical to eliminate "data skew," as TV episodes and obscure short films inherently receive much higher baseline ratings from niche audiences than mainstream cinema.

10.2 Q2: Movie Runtime Evolution

Question: Has the average movie runtime changed by decade, and does runtime correlate with rating?
Model/Query Used: `q2_runtime_by_decade.sql`
Result: The average feature film runtime has remained incredibly stable at around 94–97 minutes since the 1980s; however, the correlation between longer runtimes and higher ratings dropped from a strong positive in the 1940s–1970s to near zero in the modern era.
Interpretation: While the long epics of the "Golden Age" were treated as prestigious cultural events that audiences deeply appreciated, modern audiences do not systematically reward a movie with higher ratings simply because it has a longer runtime.

10.3 Q3: Hidden Gems vs. Overrated Titles

Question: Which genres have the highest ratio of "hidden gems" (high rating, low vote count) versus "overrated" titles (low rating, high vote count)?
Model/Query Used: `q3_hidden_gems.sql`
Result: Documentary, History, and War genres have the highest ratio of hidden gems, whereas Sci-Fi, Horror, and Action genres have the worst ratios, making them the most overrated.
Interpretation: Niche genres attract passionate audiences who consistently rate them highly, whereas mass-market blockbusters attract huge vote counts due to marketing but frequently fail to deliver on quality, resulting in mediocre ratings.

11 Challenges and Lessons Learned

1. **Handling IMDb's Custom Null Representations (\N)**
   A significant data quality issue arose during the transformation phase due to IMDb's formatting choices. The raw TSV files use the literal string `"\N"` to represent missing data. Attempting to cast numeric columns (such as `runtimeMinutes` or `startYear` to integer types) directly in DuckDB or dbt caused the pipeline to crash with type mismatch errors.
   Resolution: By wrapping the affected columns in a `NULLIF(column_name, '\N')` function before applying the `CAST()` operation, the pipeline safely converts these string literals into true SQL `NULL` values, preventing runtime failures.
2. **Data Skew and "Noise" in Analytical Aggregations**
   When initially calculating the top directors by average rating, the results were heavily skewed. Directors of obscure short films or highly-rated television anime episodes consistently ranked above established cinema directors due to the inherently different voting behaviors across media types.
   Resolution: I had to refine the business logic in the final analytical queries. By explicitly joining the `staging` metadata and enforcing a strict `title_type`

`= 'movie'` filter, the noise was successfully eliminated, yielding accurate insights.

3. **Challenge: Schema Inconsistencies During Parquet Migration**
When migrating the orchestration to Airflow, the extraction architecture was simultaneously updated to convert the raw TSV files into Parquet format before loading them into DuckDB. Following this change, downstream dbt tasks (such as `dbt_run_staging` and `dbt_snapshot`) began failing intermittently, as seen in the Airflow Grid View. These seemingly random crashes occurred because DuckDB infers schemas differently from Parquet files compared to raw TSV reads, exposing new type-casting inconsistencies that the initial dbt models were not prepared for.
Resolution: The pipeline was stabilized by enforcing much stricter SQL `CAST` operations directly within the dbt `staging` layer. Instead of altering the ingestion script, the staging models were updated to rigorously cast and handle the inferred Parquet data types, ensuring all downstream models received a uniform and predictable schema.